

VI. CONCLUSION

This paper presented GEX, an open-source platform for dexterous manipulation research. The platform integrates the 16-DoF fully actuated GX16 dexterous hand with the 16-DoF EX16 exoskeleton glove. It also provides a software stack for motion capture, retargeting, control, data collection, and real-hardware deployment. GX16 combines an actively actuated lateral palm DoF with independent finger flexion–extension and abduction–adduction. The palm DoF adjusts the spatial contact layout of the four fingers as a group. This design improves contact adaptability in enveloping grasping, in-hand reorientation, twisting, and tool manipulation. To address the morphological differences between EX16 and GX16, we formulate retargeting in a shared fingertip workspace, train a neural inverse-kinematics model through a differentiable IK-FK cycle, and introduce a Jacobian-Cycle regularizer to improve local displacement consistency. Across four real-world teleoperation tasks, GEX achieves 15/20, 18/20, 19/20, and 11/20 successful trials, respectively, corresponding to success rates of 75%, 90%, 95%, and 55%, with an average success rate of 79%. Under the same test settings, it outperforms the evaluated retargeting baselines. Demonstrations collected with EX16 further support imitation-learning tasks including plug insertion and stacked-object grasping, while policies trained in simulation transfer to the real GX16 hand for in-hand reorientation and bulb twisting. These experiments demonstrate the use of GEX for dexterous teleoperation and robot skill learning. Future work will target finer-grained dexterous manipulation, extending tactile and force-feedback capabilities and exploiting the lateral palm motion for tasks that demand precise contact placement and in-hand object control.

REFERENCES

- [1] A. Billard and D. Kragic, “Trends and challenges in robot manipulation,” *Science*, vol. 364, no. 6446, p. eaat8414, 2019.
- [2] I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas *et al.*, “Solving Rubik’s cube with a robot hand,” *arXiv preprint arXiv:1910.07113*, 2019.
- [3] C. Chi, Z. Xu, C. Pan, E. Cousineau, B. Burchfiel, S. Feng, R. Tedrake, and S. Song, “Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots,” *arXiv preprint arXiv:2402.10329*, 2024.
- [4] O. M. Andrychowicz, B. Baker, M. Chociej, R. Jozefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray *et al.*, “Learning dexterous in-hand manipulation,” *The International Journal of Robotics Research*, vol. 39, no. 1, pp. 3–20, 2020.
- [5] X. Liu, Y. Dong, J. Wan, L. Deng, F. Hua, Y. Shen, M. Yu, G. Ma, C. Cheng, H. Song, H. Ding, and Y. Yuan, “DexAnyTwist: Learning general dexterous twisting with hybrid manipulation system identification,” *National Science Review*, vol. 13, no. 8, 2026.
- [6] Z. Sun, X. Mao, J. Zhu, Y. Zhang, and R. B. Fisher, “Suction Leap-Hand: Suction cups on a multi-fingered hand enable embodied dexterity and in-hand teleoperation,” *arXiv preprint arXiv:2509.20646*, 2025.
- [7] Shadow Robot Company, “Shadow dexterous hand and shadow glove,” [Online]. Available: <https://shadowrobot.com/shadow-hand-glove/>.
- [8] SimLab Robotics, “Allegro hand ROS v4,” [Online]. Available: https://github.com/simlabrobotics/allegro_hand_ros_v4.
- [9] K. Shaw, A. Agarwal, and D. Pathak, “LEAP hand: Low-cost, efficient, and anthropomorphic hand for robot learning,” *arXiv preprint arXiv:2309.06440*, 2023.
- [10] Z. Wan, Z. Bi, Z. Zhou, H. Ren, Y. Zeng, Y. Li, L. Qi, X. Yang, M.-H. Yang, and H. Cheng, “RAPID hand: A robust, affordable, perception-integrated, dexterous manipulation platform for generalist robot autonomy,” *arXiv preprint arXiv:2506.07490*, 2025.
- [11] B. A. Richardson, F. Grüninger, L. Mack, J. Stueckler, and K. J. Kuchenbecker, “ISyHand: A dexterous multi-finger robot hand with an articulated palm,” *arXiv preprint arXiv:2509.26236*, 2025.
- [12] Z. Si, K. L. Zhang, Z. Temel, and O. Kroemer, “Tilde: Teleoperation for dexterous in-hand manipulation learning with a DeltaHand,” in *Proc. Robotics: Science and Systems (RSS)*, 2024.
- [13] H. Zhang, S. Hu, Z. Yuan, and H. Xu, “DOGlove: Dexterous manipulation with a low-cost open-source haptic force feedback glove,” *arXiv preprint arXiv:2502.07730*, 2025.
- [14] Q. Ye, Q. Liu, S. Wang, J. Chen, Y. Cui, K. Jin, H. Chen, X. Cai, G. Li, and J. Chen, “Visual-tactile pretraining and online multitask learning for humanlike manipulation dexterity,” *Science Robotics*, vol. 11, no. 110, p. eady2869, 2026.
- [15] M. Wuthrich, F. Widmaier, F. Grimmering, S. Joshi, V. Agrawal, B. Hammoud, M. Khadiv, M. Bogdanovic, V. Berenz, J. Viereck, M. Naveau, L. Righetti, B. Schölkopf, and S. Bauer, “Trifinger: An open-source robot for learning dexterity,” in *Proceedings of the 2020 Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 155. PMLR, 2021, pp. 1871–1882. [Online]. Available: <https://proceedings.mlr.press/v155/wuthrich21a.html>
- [16] J. Yuan, H. Zhu, J. Dai, and S. Yi, “Development of the bioinspired tendon-driven DexHand 021 with proprioceptive compliance control,” *arXiv preprint arXiv:2511.03481*, 2025.
- [17] L. Deng, Y. Shen, Y. Hong, Y. Dong, X. He, Y. Yuan, Z. Li, and H. Ding, “Sen-Glove: A lightweight wearable glove for hand assistance with soft joint sensing,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2022, pp. 5170–5175.
- [18] H.-S. Fang, B. Romero, Y. Xie, A. Hu, B.-R. Huang, J. Alvarez, M. Kim, G. Margolis, K. Anbarasu, M. Tomizuka, E. Adelson, and P. Agrawal, “DEXOP: A device for robotic transfer of dexterous human manipulation,” *arXiv preprint arXiv:2509.04441*, 2025.
- [19] A. Handa, K. Van Wyk, W. Yang, J. Liang, Y.-W. Chao, Q. Wan, S. Birchfield, N. Ratliff, and D. Fox, “Dexpilot: Vision-based teleoperation of dexterous robotic hand-arm system,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 9164–9170.
- [20] Y. Qin, W. Yang, B. Huang, K. Van Wyk, H. Su, X. Wang, Y.-W. Chao, and D. Fox, “Anyteleop: A general vision-based dexterous robot arm-hand teleoperation system,” *arXiv preprint arXiv:2307.04577*, 2023.
- [21] A. Iyer, Z. Peng, Y. Dai, I. Guzey, S. Halder, S. Chintala, and L. Pinto, “OPEN TEACH: A versatile teleoperation system for robotic manipulation,” *arXiv preprint arXiv:2403.07870*, 2024.
- [22] Y. Qin, Y.-H. Wu, S. Liu, H. Jiang, R. Yang, Y. Fu, and X. Wang, “Dexmv: Imitation learning for dexterous manipulation from human videos,” in *Computer Vision – ECCV 2022*, ser. Lecture Notes in Computer Science, vol. 13699. Springer, 2022, pp. 570–587.
- [23] Z.-H. Yin, C. Wang, L. Pineda, F. Hogan, K. Bodduluri, A. Sharma, P. Lancaster, I. Prasad, M. Kalakrishnan, J. Malik, M. Lambeta, T. Wu, P. Abbeel, and M. Mukadam, “DexterityGen: Foundation controller for unprecedented dexterity,” in *Proc. Robotics: Science and Systems (RSS)*, 2025.
- [24] C. Chen, Z. Yu, H. Choi, M. Cutkosky, and J. Bohg, “DexForce: Extracting force-informed actions from kinesthetic demonstrations for dexterous manipulation,” *arXiv preprint arXiv:2501.10356*, 2025.
- [25] J. Wan, X. Liu, and Y. Dong, “DexReMoE: In-hand reorientation of general object via mixtures of experts,” *arXiv preprint arXiv:2508.01695*, 2025.
- [26] Z.-H. Yin, C. Wang, L. Pineda, K. Bodduluri, T. Wu, P. Abbeel, and M. Mukadam, “Geometric retargeting: A principled, ultrafast neural hand retargeting algorithm,” *arXiv preprint arXiv:2503.07541*, 2025.
- [27] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [28] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-real transfer of robotic control with dynamics randomization,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [29] M. Mittal, P. Roth, J. Tigue, A. Richard, O. Zhang, P. Du *et al.*, “Isaac lab: A GPU-accelerated simulation framework for multi-modal robot learning,” *arXiv preprint arXiv:2511.04831*, 2025.

APPENDIX

HARDWARE BILL OF MATERIALS

Table IV lists the parts required to build one GX16 dexterous hand and one EX16 exoskeleton glove, for a total parts cost of \$1,420 per complete hand–glove system. The quantities follow directly from the actuation scheme described in Section III: 15 finger joints plus the palm lateral-swing joint for GX16, and four finger modules with four joints each for EX16. GX16 uses the XL330-M288-T at the 15 finger joints and the metal-gearred XC330-M288-T at the palm lateral-swing joint, which must carry the additional torque of driving the whole palm and four fingers; EX16 uses the lower-friction XL330-M077-T at all 16 joints so that the operator’s fingers can backdrive the exoskeleton. A complete hand–glove system therefore requires 32 actuators, which account for \$560 of the GX16 cost and \$480 of the EX16 cost, so the two devices remain comparable in price despite this difference. Structural parts are printed in-house from engineering filament, and all prices are quoted in US dollars at manufacturer list price excluding tax.

TABLE IV: Bill of materials for one GX16 dexterous hand and one EX16 exoskeleton glove. Quantities are per device. The subtotal column accumulates down the table, so the final row gives the cost of a complete hand–glove system.

Device	Category	Component	Qty	Unit (\$)	Subtotal (\$)
GX16	Actuator	Dynamixel XL330-M288-T, finger joints	15	30	450
	Actuator	Dynamixel XC330-M288-T, palm lateral swing	1	110	110
	Driver	OpenRB-150 controller board	1	100	100
	Electronics	Power supply, U2D2, hubs, cabling	1 set	30	30
	Structural	3D-printed parts (filament)	1 set	40	40
	Hardware	Bearings, screws, heat-set inserts, springs	1 set	20	20
<i>GX16 subtotal</i>					750
EX16	Actuator	Dynamixel XL330-M077-T, finger joints	16	30	480
	Driver	OpenRB-150 controller board	1	100	100
	Electronics	Power supply, U2D2, hubs, cabling	1 set	30	30
	Structural	3D-printed palm frame, finger sleeves, linkages	1 set	40	40
	Hardware	Bearings, screws, heat-set inserts, straps	1 set	20	20
<i>EX16 subtotal</i>					670
<i>Complete hand–glove system</i>					1,420

RETARGETING IMPLEMENTATION DETAILS

This appendix records the parameterization, numerical properties, and validity boundary of the retargeting formulation summarized in Section IV-A.

A. Problem Setup and Parameterization

Human and robot hands may have different joint spaces while still producing corresponding fingertip configurations, so the retargeting problem is posed in a shared fingertip space rather than by matching joint angles directly. Let $\mathbf{p} \in \mathbb{R}^{3M}$ denote the aligned fingertip target, where M is the number of fingertips, and let n denote the number of robot joints. The neural inverse-kinematics network G_θ maps this target to normalized joint variables, the decoder D maps those variables to physical joint angles, and the frozen forward-kinematics surrogate F_ϕ closes the differentiable loop $\mathbf{h}_\theta = F_\phi \circ D \circ G_\theta$.

Fingertip targets enter this loop after a morphology-alignment transform that maps human fingertip coordinates into the robot palm frame. The shared variant of this transform is

$$C(\mathbf{x}) = s\mathbf{R}\mathbf{x} + \mathbf{t}, \quad (6)$$

where $s > 0$ is a scale, $\mathbf{R} \in SO(3)$ is a rotation, and $\mathbf{t} \in \mathbb{R}^3$ is a translation. A single global transform cannot absorb per-finger length and base-frame differences across hands, so our experiments also consider local calibration variants that assign a separate translation, scale, or rotation to each finger. The choice among these variants is treated as a per-hand calibration setting rather than as a fixed part of the formulation.

On the output side, the network predicts a normalized joint variable z_i for each joint i and decodes it into a physical joint angle through the following affine map:

$$q_i = \frac{q_i^{\max} - q_i^{\min}}{2} z_i + \frac{q_i^{\max} + q_i^{\min}}{2}, \quad (7)$$

where $[q_i^{\min}, q_i^{\max}]$ is the nominal joint interval of joint i . This map rescales the normalized output into the angular range spanned by the bounds.

B. Differentiable IK–FK Loop and Losses

The retargeting network predicts normalized joints and reconstructs fingertip geometry through a frozen forward-kinematics surrogate:

$$\mathbf{z} = G_\theta(\mathbf{p}), \quad \mathbf{q} = D(\mathbf{z}), \quad \mathbf{h}_\theta(\mathbf{p}) = F_\phi(D(G_\theta(\mathbf{p}))). \quad (8)$$

The point-reconstruction loss is

$$\mathcal{L}_P = \frac{1}{BM} \sum_{b=1}^B \left\| \mathbf{h}_\theta(\mathbf{p}^{(b)}) - \mathbf{p}^{(b)} \right\|_2^2. \quad (9)$$

For samples with robot joint labels, sparse normalized anchors are added using a Smooth L1 penalty:

$$\mathcal{L}_A = \frac{1}{N_A n} \sum_{a=1}^{N_A} \sum_{j=1}^n \ell_\beta ([G_\theta(\mathbf{p}_a)]_j - [\mathbf{z}_a]_j). \quad (10)$$

These two terms constrain the value of the loop at sampled configurations, but they do not by themselves constrain how the reconstruction changes between samples, so a small input displacement may be reproduced with an incorrect direction or magnitude even when the final point error is small. The Jacobian-Cycle term addresses this zero-order limitation by constraining the first-order local response of the complete IK–FK loop toward the identity,

$$\mathbf{J}_h(\mathbf{p}) \approx \mathbf{I}_{3M}, \quad (11)$$

so that a displacement applied to the human fingertip target reappears as the same displacement of the reconstructed robot fingertips.

For a single fingertip, the input and reconstructed output are three-dimensional and the local Jacobian is a 3×3 matrix. For M fingertips, $\mathbf{J}_h$ is a $3M \times 3M$ block matrix whose off-diagonal blocks describe the motion of one fingertip caused by perturbing another, so the identity target also suppresses unintended cross-finger coupling. The full Jacobian objective would be $\|\mathbf{J}_h(\mathbf{p}) - \mathbf{I}_{3M}\|_F^2$, but explicitly forming this matrix for every sample is computationally expensive. For a sufficiently smooth loop, the chain rule decomposes this matrix into the three stages of the cycle,

$$\mathbf{J}_h(\mathbf{p}) = \mathbf{J}_F(D(G_\theta(\mathbf{p})))\mathbf{J}_D(G_\theta(\mathbf{p}))\mathbf{J}_G(\mathbf{p}), \quad (12)$$

where $\mathbf{J}_G$ is the local response of the inverse-kinematics network, $\mathbf{J}_D$ is the joint-range decoder Jacobian, and $\mathbf{J}_F$ describes the local forward-kinematics variation. Thus, the Jacobian-Cycle constraint encourages the local response of the IK network and the robot geometry to compose into an approximately identity map.

The four terms in the objective therefore play complementary roles. The point-reconstruction term answers whether an input fingertip configuration returns to the same position after the IK–FK cycle, but it does not impose a unique joint solution, because multiple joint configurations may produce nearly identical fingertip positions. The sparse anchor term supplies a weak joint-space prior only at paired configurations and reduces this ambiguity without replacing fingertip-space supervision. The direction loss primarily constrains the direction of a finite displacement, whereas the Jacobian-Cycle loss constrains both local direction and magnitude. Together, point reconstruction, direction, sparse anchors, and Jacobian-Cycle emphasize position, displacement direction, joint-space disambiguation, and first-order local consistency, respectively.

C. Jacobian–Cycle Estimation

The loop Jacobian is $\mathbf{J}_h(\mathbf{p}) = \partial \mathbf{h}_\theta(\mathbf{p}) / \partial \mathbf{p}$. Rather than forming the full matrix, we sample L normalized directions $\mathbf{v}^{(b,\ell)}$ per input and probe the loop along each of them with a central difference,

$$\hat{\mathbf{d}}^{(b,\ell)} = \frac{\mathbf{h}_\theta(\mathbf{p}^{(b)} + \varepsilon \mathbf{v}^{(b,\ell)}) - \mathbf{h}_\theta(\mathbf{p}^{(b)} - \varepsilon \mathbf{v}^{(b,\ell)})}{2\varepsilon}. \quad (13)$$

The finite-difference Jacobian loss is

$$\mathcal{L}_J^{\text{FD}} = \frac{1}{BLd} \sum_{b=1}^B \sum_{\ell=1}^L \left\| \hat{\mathbf{d}}^{(b,\ell)} - \mathbf{v}^{(b,\ell)} \right\|_2^2, \quad d = 3M. \quad (14)$$

The complete objective is $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{base}} + \lambda_J \mathcal{L}_J^{\text{FD}} + \lambda_A \mathcal{L}_A$, where $\mathcal{L}_{\text{base}}$ contains the point reconstruction term and any selected geometric terms.

During training, the forward-kinematics surrogate F_ϕ is frozen so that its parameters are not updated, but the computation graph must remain connected through F_ϕ to G_θ , because detaching either probe output would remove the Jacobian-Cycle gradient needed to update the inverse-kinematics network.

D. Validity Boundary and Numerical Choices

The rank of $\mathbf{J}_h$ is at most $\min(3M, n)$, and it is further restricted by joint limits, kinematic singularities, and the reachable workspace. When the task-space dimension exceeds the available joint degrees of freedom, a full identity Jacobian is generally unattainable in all directions. The identity target in (11) must therefore be read as a soft regularizer rather than as a hard constraint that holds for every input and every direction.

The random-direction estimator inherits the usual properties of central differences: the truncation error decays quadratically with the step ε when the mapping is sufficiently smooth, and the estimator is proportional in expectation to the full Jacobian objective. The finite-difference step ε is expressed in meters and should be large enough to avoid floating-point noise while remaining small enough to represent a local perturbation. Increasing L improves directional coverage but increases the computation approximately linearly because each direction requires two additional loop evaluations.

The two losses also differ in units: the Jacobian-Cycle loss is dimensionless, whereas the point-reconstruction loss has units of squared meters. The weight λ_J should therefore be selected with respect to the numerical scale of the complete objective rather than copied from the point-loss weight. A large Jacobian weight may overconstrain local scale or cross-finger coupling and reduce workspace coverage or joint naturalness, which sets the upper end of the usable range together with the rank limitation above.

E. Ablation Protocol

This subsection records the settings behind the ablation reported in Table III, where each configuration is trained and evaluated separately on the Allegro Hand, LEAP Hand, Shadow Hand, and GX16. Operator hand trajectories are recorded with EX16 and split chronologically per source, using the first 80% for training and the last 20% for testing, giving 9,600 training frames and 2,400 test frames in total. All configurations use random seed 260808, point-cycle weight 50, finite-difference step $\varepsilon = 1$ mm, a single probe direction per sample ($L = 1$), Jacobian weight $\lambda_J = 0.01$, and no anchor loss ($\lambda_A = 0$). The local-alignment variant differs per hand: per-finger palm rotation for the Allegro Hand, per-finger offset for the Shadow Hand, and per-finger scale for the LEAP Hand and GX16.

The reported metric is the mean Euclidean distance between the URDF forward-kinematics fingertip position and the human target fingertip position, averaged over all test frames and all valid fingertips:

$$\bar{d} = \frac{1}{TF} \sum_{t=1}^T \sum_{f=1}^F \|\mathbf{p}_{t,f}^{\text{urdf}} - \mathbf{p}_{t,f}^{\text{human}}\|_2. \quad (15)$$

Because this metric is recomputed with the real robot URDF, it does not depend on the accuracy of the neural forward-kinematics surrogate F_ϕ used during training. Fig. 8 shows retargeted configurations for the four hands on representative operator poses from the same test split.

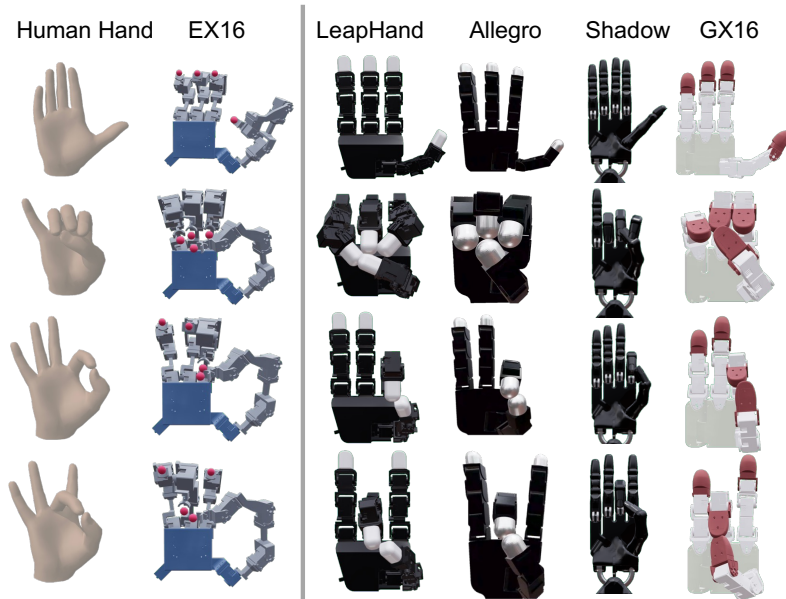


Fig. 8: Qualitative cross-hand retargeting examples. Each row pairs a representative human-hand pose and the corresponding EX16 input configuration with retargeted configurations for LeapHand, AllegroHand, ShadowHand, and GX16.

F. Fingertip Opposition Space Generation

The procedure described here is an offline module for generic multi-finger robot models rather than a GX16-specific construction. The five-finger configuration below illustrates the general formulation; for GX16, the same procedure is instantiated with its four-finger model and corresponding URDF. The links and fingertip pads of a dexterous hand have finite volumes, so fingertip opposition must be defined between physical collision surfaces rather than idealized fingertip points. Moreover, a specified thumb–finger opposition is generally one-to-many in joint space, because multiple robot configurations can realize the same near-contact relationship while satisfying the joint limits and self-collision constraints. We therefore construct a discrete opposition set for a five-finger robot hand offline to retain these geometrically feasible alternatives for the retargeting framework. Fig. 9 illustrates this multiplicity with several representative solutions for thumb–index, thumb–middle, and thumb–ring opposition.

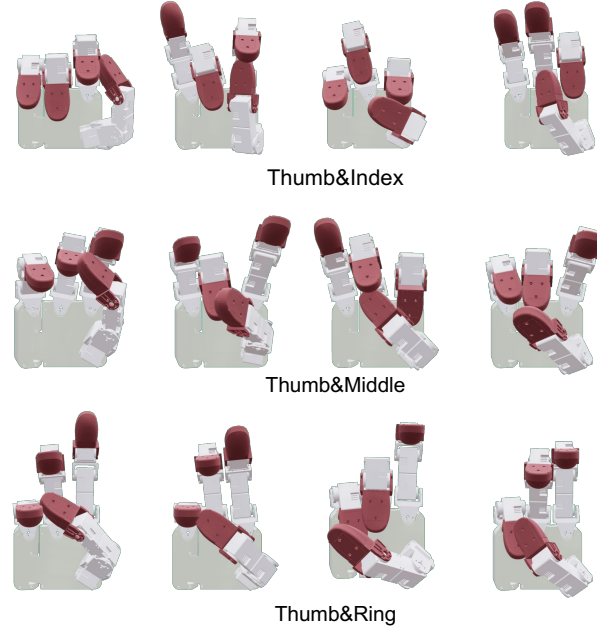


Fig. 9: Representative fingertip-opposition configurations generated by the offline sampling procedure for the thumb–index, thumb–middle, and thumb–ring pairs.

Let the robot configuration, finger set, and candidate thumb–finger pairs be

$$\begin{aligned} \mathbf{q}_r &\in \mathcal{Q}_r = [\mathbf{q}_{\min}, \mathbf{q}_{\max}] \subset \mathbb{R}^{n_r}, \\ \mathcal{F} &= \{\text{thumb}, \text{index}, \text{middle}, \text{ring}, \text{little}\}, \\ \mathcal{P}_{\text{pinch}} &= \{(\text{thumb}, j) \mid j \in \mathcal{F} \setminus \{\text{thumb}\}\}. \end{aligned} \quad (16)$$

For each finger $i \in \mathcal{F}$, its terminal collision link ℓ_i is read from the robot URDF, as summarized with the remaining implementation parameters in Table V.

For a candidate pair $\varpi = (i, j) \in \mathcal{P}_{\text{pinch}}$, let $S_i(\mathbf{q}_r)$ and $S_j(\mathbf{q}_r)$ denote the two collision-mesh surfaces expressed in the palm-base frame. Their signed surface distance is defined as

$$d_{\varpi}(\mathbf{q}_r) = \begin{cases} \text{dist}(S_i(\mathbf{q}_r), S_j(\mathbf{q}_r)), & \text{if the surfaces are separated,} \\ -\delta_{\varpi}(\mathbf{q}_r), & \text{if penetration is detected,} \end{cases} \quad (17)$$

where $\delta_{\varpi}(\mathbf{q}_r)$ is the penetration depth returned by SAPIEN/PhysX. Thus, positive, zero, and negative values indicate separation, contact, and mesh penetration, respectively.

We define the sampled opposition set as

$$\hat{\mathcal{Q}}_{\text{pinch}} = \{\mathbf{q}_r \in \mathcal{Q}_{\text{safe}} \mid \exists \varpi \in \mathcal{P}_{\text{pinch}}, d_{\min} \leq d_{\varpi}(\mathbf{q}_r) \leq d_{\max}\}, \quad (18)$$

where $d_{\min}$ and $d_{\max}$ are the lower and upper signed-distance thresholds, respectively. The safe set $\mathcal{Q}_{\text{safe}}$ permits penetration up to δ_{target} for the intended contact pairs and up to δ_{other} for every other self-collision pair. This asymmetric tolerance accommodates numerical penetration at the intended pinch while excluding substantial collisions involving non-target links.

The final acceptance test uses collision-mesh surfaces rather than Euclidean distances between fixed fingertip keypoints, because the keypoint frames can be offset from the physical contact surfaces. Each terminal mesh is sampled deterministically at two resolutions, using N_{coarse} surface points for coarse screening and N_{fine} points for final verification. For separated meshes, the procedure evaluates the bidirectional sampled point-to-surface distance and takes the smaller minimum; the fine stage shortlists K_{fine} candidates with a KD-tree before point-to-triangle closest-point queries.

Seed configurations are first sought in a base set $\mathcal{D}_0$ containing N_0 uniformly sampled joint configurations that have passed self-collision filtering. A loose keypoint test, $d_{\text{key}}(\mathbf{q}_r) \leq d_{\text{max}} + \Delta_{\text{key}}$, is used only to reduce the number of expensive mesh-distance evaluations. The implementation examines at most N_{screen} base configurations and retains up to N_{seed} seeds after applying the complete surface-distance and collision tests. If no seed passes these tests, a bounded Powell search is initialized from the base configuration $\mathbf{q}_0$ with the smallest keypoint distance and minimizes

$$\min_{\mathbf{q}_r \in \mathcal{Q}_r} d_{\text{near}}(\mathbf{q}_r)^2 + \lambda_{\text{seed}} \left\| \frac{\mathbf{q}_r - \mathbf{q}_0}{\mathbf{q}_{\text{max}} - \mathbf{q}_{\text{min}}} \right\|_2^2, \quad (19)$$

where $d_{\text{near}}(\mathbf{q}_r) = \min_{\varpi \in \mathcal{P}_{\text{pinch}}} d_{\varpi}(\mathbf{q}_r)$. An optimized configuration is accepted as a seed only after it passes the same complete validation tests.

The feasible region is expanded by adaptive local rejection sampling around the accepted pool $\mathcal{R}$. For a base configuration $\mathbf{q}_b$ drawn from $\mathcal{R}$, a candidate is generated by

$$\tilde{\mathbf{q}}_r = \text{clip}(\mathbf{q}_b + \sigma \epsilon \odot (\mathbf{q}_{\text{max}} - \mathbf{q}_{\text{min}}), \mathbf{q}_{\text{min}}, \mathbf{q}_{\text{max}}), \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (20)$$

where $\odot$ denotes elementwise multiplication and σ is initialized to σ_0 . Scaling by the joint ranges gives joints with different motion limits comparable relative exploration amplitudes. Each candidate is clipped to the joint limits and then subjected, in order, to self-collision checking, coarse mesh-distance screening, and fine mesh-distance verification. Accepted candidates are added both to $\hat{\mathcal{Q}}_{\text{pinch}}$ and to the sampling pool, which retains at most N_{pool} configurations.

After every N_{adapt} proposals, the perturbation scale is updated using the recent acceptance rate ρ :

$$\sigma \leftarrow \begin{cases} \max(\gamma_{\downarrow} \sigma, \sigma_{\text{min}}), & \rho < \rho_{\text{low}}, \\ \min(\gamma_{\uparrow} \sigma, \sigma_{\text{max}}), & \rho > \rho_{\text{high}}, \\ \sigma, & \text{otherwise.} \end{cases} \quad (21)$$

A low acceptance rate therefore contracts the search around the feasible set, whereas a high rate increases coverage by enlarging the perturbation scale. All implementation constants are collected in Table V.

TABLE V: Implementation parameters for five-finger opposition-space generation.

Parameter	Value	Description
$ \mathcal{F} $	5	Number of fingers
ℓ_i	Robot-dependent	Terminal collision link read from the URDF
d_{min}	−10 mm	Minimum signed surface distance
d_{max}	5 mm	Maximum signed surface distance
δ_{target}	10 mm	Target-pair penetration limit
δ_{other}	0.5 mm	Non-target penetration limit
N_{coarse}	128	Coarse surface samples per mesh
N_{fine}	2048	Fine surface samples per mesh
K_{fine}	32	Fine-stage KD-tree shortlist size
N_0	100,000	Size of collision-free base set
Δ_{key}	60 mm	Keypoint pre-screening margin
N_{screen}	20,000	Maximum screened base configurations
N_{seed}	32	Maximum retained seeds
λ_{seed}	10^{-5}	Seed-search regularization weight
σ_0	0.03	Initial relative perturbation scale
N_{pool}	10,000	Maximum sampling-pool size
N_{adapt}	500	Scale-update interval
$\rho_{\text{low}}, \rho_{\text{high}}$	0.1, 0.5	Acceptance-rate thresholds
$\gamma_{\downarrow}, \gamma_{\uparrow}$	0.8, 1.15	Scale contraction and expansion factors
$\sigma_{\text{min}}, \sigma_{\text{max}}$	0.002, 0.15	Perturbation-scale bounds

Algorithm 1 summarizes the complete procedure. The resulting data constitute an adaptively sampled near-contact pinch set rather than a volume-uniform sample of the n_r -dimensional joint space.

The task-specific manipulation spaces used in our framework are learned through reinforcement learning in simulation. Separate policies are trained for bulb twisting and in-hand reorientation using the direct reinforcement-learning environments implemented in NVIDIA Isaac Lab. Fig. 10 provides an overview of the two task-specific simulation environments.

Algorithm 1 Adaptive generation of a five-finger opposition set.

Require: Five-finger robot-hand model, collision-free base set $\mathcal{D}_0$, target size N_{pinch}

Ensure: Sampled opposition set $\hat{\mathcal{Q}}_{\text{pinch}}$

- 1: Load joint limits and terminal collision meshes; construct coarse and fine surface samples
 - 2: Screen $\mathcal{D}_0$ with the loose keypoint condition and validate the retained configurations
 - 3: **if** no valid seed is found **then**
 - 4: Obtain a candidate seed by solving (19) and apply complete validation
 - 5: **end if**
 - 6: Initialize $\mathcal{R}$ and $\hat{\mathcal{Q}}_{\text{pinch}}$ with the valid seeds; set $\sigma \leftarrow \sigma_0$
 - 7: **while** $|\hat{\mathcal{Q}}_{\text{pinch}}| < N_{\text{pinch}}$ **do**
 - 8: Draw $\mathbf{q}_b$ from $\mathcal{R}$ and generate $\tilde{\mathbf{q}}_r$ using (20)
 - 9: Evaluate non-target self-collisions and target-pair penetration limits
 - 10: Apply coarse screening and fine signed-distance verification
 - 11: **if** $\tilde{\mathbf{q}}_r$ satisfies (18) **then**
 - 12: Add $\tilde{\mathbf{q}}_r$ to $\hat{\mathcal{Q}}_{\text{pinch}}$ and update $\mathcal{R}$
 - 13: **end if**
 - 14: Every N_{adapt} proposals, update σ using (21)
 - 15: **end while**
 - 16: Store joint angles, fingertip positions, contact labels, signed distances, and closest surface points
-

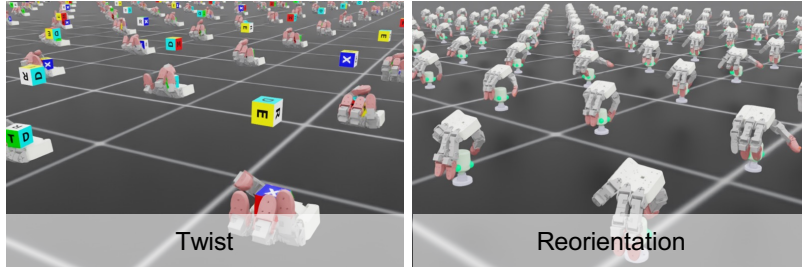


Fig. 10: Isaac Lab simulation environments used to train the task-specific bulb twisting and in-hand reorientation policies.

All two environments use the 16 actuated joints of the four-finger GX16 hand, and each policy outputs a bounded joint-target action $\mathbf{a}_t \in [-1, 1]^{n_r}$. The normalized action is mapped to the corresponding joint limits and temporally filtered before being sent to the position-controlled actuators.

We optimize all policies with PPO using 16 rollout steps per environment and an upper limit of 10,000 training iterations. The shared PPO settings are a learning rate of 5×10^{-4} with an adaptive schedule, a discount factor of 0.99, a generalized-advantage-estimation factor of 0.95, a clipping ratio of 0.2, and an entropy coefficient of 0.005. Each update uses five learning epochs and four mini-batches, with a desired KL divergence of 0.016 and a maximum gradient norm of 1.0. Actor and critic observations are normalized, and the initial standard deviation of the Gaussian action distribution is 1.0. Table VI summarizes the task-dependent simulation and network configurations.

TABLE VI: Task-dependent reinforcement-learning configurations.

Configuration	Bulb twisting	In-hand reorientation
Parallel environments	64	1024
Physics/control frequency	120/60 Hz	60/30 Hz
Maximum episode duration	10 s	20 s
Recurrent layer	None	One-layer LSTM with 256 hidden units
MLP hidden layers	[512, 512, 256, 128]	[512, 256, 128]
Activation	ELU	ELU
Target moving-average coefficient	0.2	0.5

For in-hand reorientation, the actor uses the policy observations listed below, whereas the critic additionally receives fingertip contact forces, the sampled domain-randomization state, and the external object force. The bulb twisting policy uses the same observation vector for action and value estimation. The domain-randomization distributions and training disturbances used for in-hand reorientation are reported separately in Section I.

G. Bulb Twisting

The bulb twisting task requires the hand to rotate an articulated object about its main rotational joint while keeping the two auxiliary rotational coordinates near zero, so the object is turned rather than displaced or tilted. Because the object class and geometry vary across episodes, the observation includes an object-class index and a geometry feature, and the action is expressed as a residual around an object-specific mean configuration. The observation, action, reward, and termination definitions are as follows.

- 1) **Observation.** The observation contains normalized joint positions, the previous joint-target action, fingertip positions, a normalized object-class index, a 128-dimensional object geometry feature, the three object-joint coordinates, the desired rotation-axis sign, and the object keypoint positions.
- 2) **Action.** The policy predicts a residual around an object-specific mean hand configuration, after which the combined action is clipped, perturbed during training, scaled to the joint limits, and filtered before execution.
- 3) **Reward.** The active reward terms are the main rotational-joint coordinate, a squared action penalty with weight -10^{-4} , a nearest-keypoint distance term with weight -50 , and a positive alignment term that decreases with the summed magnitudes of the two auxiliary rotational-joint coordinates and is capped at 2. Fingertip-to-object-keypoint distances below 0.03 m are assigned zero cost, and reaching a main-joint rotation greater than 4 rad yields a success bonus of 1000. The progress-related terms present in the implementation have zero weights in the supplied training configuration and are therefore omitted from the active objective above.
- 4) **Termination.** After a success, the object joints are reset while the hand configuration and episode phase are retained, allowing repeated twisting motions within one rollout. The episode terminates if either auxiliary rotation exceeds 0.15 rad in magnitude and is otherwise truncated after 10 s.

H. In-Hand Reorientation

The in-hand reorientation task requires the hand to rotate a grasped object toward a commanded orientation without dropping it, which makes the episode a sequence of contact transitions rather than a single grasp. The policy therefore uses a recurrent layer and a larger number of parallel environments than bulb twisting, as listed in Table VI. The observation, action, reward, and termination definitions are as follows.

- 1) **Observation.** The actor observation concatenates normalized joint positions, scaled joint velocities, object pose and velocity, the hand reference position, the target orientation, the relative object-to-target orientation, fingertip poses and spatial velocities, and the preceding action.
- 2) **Action.** The normalized action is scaled to the joint limits, optionally mixed with a random-network perturbation during training, and filtered before being applied as the joint-position target.
- 3) **Reward.** The reward includes the object-to-hand-reference distance with weight -10 , the reciprocal orientation error with numerator 1 and offset 0.1, and the squared action magnitude with weight -2×10^{-4} . An orientation error no greater than 0.25 rad yields a success bonus of 500, whereas an object-to-hand-reference distance of at least 0.24 m incurs a penalty of -5 .
- 4) **Termination.** Target orientations are sampled from rotations in 90° increments about the three Cartesian axes, and a new target is sampled immediately after each success without ending the episode. The episode terminates when the object-to-hand-reference distance reaches 0.24 m and is otherwise truncated after 20 s.

I. Domain Randomization

Policies trained with fixed nominal simulation parameters can become overly dependent on a particular dynamics and sensing model, making their behavior sensitive to modeling errors, calibration uncertainty, and unmodeled disturbances. This issue is particularly important for in-hand reorientation because repeated contact transitions couple object properties, friction, robot kinematics, and actuator behavior throughout the task. Domain randomization exposes the policy to a distribution of plausible training conditions, reducing its reliance on exact simulator parameters and supporting robustness to deviations from the nominal model.

For the in-hand reorientation policy, we randomize object properties, contact parameters, robot kinematics, actuator parameters, initial states, and sensing conditions, and we additionally inject action and force disturbances during simulation training. Table VII lists each randomized quantity together with its sampling distribution or range and resampling schedule, distinguishing slowly varying model uncertainty from episode-level variation and control-step disturbances. The ranges reported for multiplicative parameters are defined relative to their nominal values.

IMITATION LEARNING SETUP

J. Policy, Demonstrations, and Observation Space

We use the $\pi_{0.5}$ vision-language-action model [27] as the imitation learning policy, predicting robot actions directly from visual observations, proprioceptive state, tactile readings, and a per-task language instruction. The policy is initialized

TABLE VII: Domain randomization and training disturbances for the in-hand reorientation task.

Category	Parameter	Distribution or range	Resampling schedule
Object geometry	Uniform scale	$s_{\text{obj}} \sim \mathcal{U}(0.82, 0.88)$	Before simulation startup
Object dynamics	Mass scale	$m_{\text{obj}}/m_{\text{obj},0} \sim \mathcal{U}(0.5, 1.5)$	At reset, with at least 720 control steps between samples
Object contact	Static friction, dynamic friction, and restitution	$\mu_s \sim \mathcal{U}(0.2, 1.3)$; $\mu_d = 1.0$; $e \sim \mathcal{U}(0, 0.4)$	At reset, with at least 720 control steps between samples
Robot contact	Static friction, dynamic friction, and restitution	$\mu_s \sim \mathcal{U}(0.7, 1.3)$; $\mu_d = 1.0$; $e \sim \mathcal{U}(0, 0.4)$	At reset, with at least 720 control steps between samples
Actuator	Joint stiffness and damping scales	$k_p/k_{p,0} \sim \text{LogUniform}(0.75, 1.5)$; $k_d/k_{d,0} \sim \text{LogUniform}(0.3, 3.0)$	At reset, with at least 720 control steps between samples
Robot kinematics	Lower and upper joint-limit offsets	$\delta q_{\min}, \delta q_{\max} \sim \mathcal{N}(0, 0.01^2)$	At reset, with at least 720 control steps between samples
Gravity	Additive gravity offset	$\Delta \mathbf{g} \sim \mathcal{N}(\mathbf{0}, \text{diag}(0, 0, 0.4^2))$	Every 24 s of global simulation time
Initial object state	Horizontal position and orientation	$\delta x_{\text{obj}}, \delta y_{\text{obj}} \sim \mathcal{U}(-0.01, 0.01)$ m; $\theta_x, \theta_y \sim \mathcal{U}(-\pi, \pi)$	Every episode reset
Initial hand state	Joint position	Interpolation by 0.2 toward a configuration sampled uniformly within the joint limits	Every episode reset
Action noise	Gaussian perturbation and bias-noise sample	$\sigma_a = 0.05$ and $\sigma_{b_a} = 0.02$	Perturbation every control step; bias at reset
Observation noise	Gaussian perturbation and bias-noise sample	$\sigma_o = 0.005$ and $\sigma_{b_o} = 0.01$	Perturbation every control step; bias at reset
Force disturbance	Object force and application probability	$\mathbf{f} \sim \mathcal{N}(\mathbf{0}, (2m_{\text{obj}})^2 \mathbf{I})$; $p_f \sim \text{LogUniform}(0.001, 0.1)$	Force trial every control step; p_f at reset
Action disturbance	Random-network-adversary action	Applied with probability 0.15 and mixed with weight $\alpha \sim \mathcal{U}(0, 1)$	Every control step; network resampled every 1000 steps

from the official $\pi_{0.5}$ pre-trained weights and fine-tuned with LoRA on our collected demonstrations: the vision-language backbone and the action expert are each updated through a low-rank adapter while all non-LoRA parameters remain frozen. This setup allows the policy to adapt to the GEX platform kinematics and observation layout at limited demonstration scale while retaining the semantic priors of the pre-trained model.

Demonstration data are collected by a human operator wearing the EX16 exoskeleton glove to teleoperate the GX16 hand; the retargeting module maps hand motions to robot joint commands, and dual-stream RGB video, robot joint states, and commands are recorded synchronously. Raw recordings are captured at 15 Hz and resampled to 10 Hz by nearest-timestamp interpolation, which is the uniform control rate used for training and deployment. Global-camera frames are cropped to a fixed region of interest that removes task-irrelevant background, while the wrist camera retains the full 640×480 RGB frame. This crop is held fixed across all training and evaluation runs, since changing it changes the effective workspace visible to the policy.

The policy observation therefore comprises two RGB streams, robot proprioception, and fingertip tactile readings. Proprioceptive state is formed by concatenating the 7-DoF arm joints with the 16-DoF GX16 hand joints, yielding a 23-dimensional vector. Each of the four fingers is equipped with 135 tactile sensing units, and each unit reports a three-axis reading, so a raw tactile frame has shape $4 \times 135 \times 3$. The tactile unit used is the PaXini ITPU GEN3 DP-L3530¹. We average over the 135 units of each finger independently per axis, obtaining a 3-dimensional contact vector per finger and a 12-dimensional tactile feature for the hand, which is concatenated with the 23-dimensional joint state to form a 35-dimensional state input. The action is defined as the 23-dimensional absolute joint target for the next control step, sharing the same joint ordering as the proprioceptive state so that the policy output can be forwarded directly to the joint-level controller without additional integration or coordinate conversion.

K. Training and Closed-Loop Deployment

The model is trained in bfloat16, with an action horizon of 24 steps (2.4 s) and a maximum text token length of 200. The internal action representation width of $\pi_{0.5}$ is fixed at 32; our 23-dimensional joint target is embedded within it via zero-padding. The optimizer is AdamW with $\beta_1=0.9$, $\beta_2=0.95$, $\epsilon=10^{-8}$, weight decay 10^{-10} , and global-norm gradient clipping at 1.0; exponential moving average is not used. The learning rate reaches a peak of 5×10^{-5} after a 1000-step linear warmup and remains at that level throughout our training budget. Training runs on a single GPU with batch size 1 for 50,000

¹<https://paxini.com/>

steps; checkpoints are saved every 1000 steps and the random seed is fixed at 42. Normalization statistics are computed once over the entire training set before training and held fixed thereafter. Table VIII summarizes the key configuration.

TABLE VIII: Key configuration of the imitation learning setup.

Category	Parameter	Value
Policy	Base model	$\pi_{0.5}$
	Fine-tuning	LoRA (backbone + expert)
	Precision	bfloat16
	Max token length	200
Observation	Global camera	244×192
	Wrist camera	640×480
	Joint state	23-D (7 arm + 16 hand)
	Tactile feature	12-D (4 fingers)
	State input	35-D (23 + 12)
	Action	23-D joint target
Temporal	Control rate	10 Hz
	Action horizon	24 (2.4 s)
	Executed per chunk	16 (1.6 s)
Optimization	Optimizer	AdamW
	Peak learning rate	5×10^{-5}
	Warmup steps	1000
	Weight decay	10^{-10}
	Gradient clipping	1.0
	Batch size	1
	Training steps	50,000
	Random seed	42

At deployment, the policy outputs an action chunk of length 24 per inference step; only the first 16 steps are executed before re-predicting from the latest observation. This receding-horizon execution keeps trajectories smooth while bounding the policy’s effective replanning period to 1.6 s, allowing it to respond to object pose changes. Each task is paired with a fixed natural-language instruction used as the language input to the policy, held constant between training and evaluation to avoid instruction wording becoming a confounding variable.